INFORME

Práctica 11 - Verificación y validación

Alumnas: Julieta Barahona Roth, Mia Bertola y Valentina Calzetta

1. Problema y objetivo

En esta práctica trabajamos con una cola de solicitudes atendidas por servidores. Retomamos el tipo de sistema de las prácticas 4 y 5, pero nos concentramos en comprobar su funcionamiento y en comparar sus resultados con una referencia.

La pregunta que nos planteamos fue: ¿cómo podemos comprobar que el simulador cumple las reglas que definimos y que sus resultados coinciden con lo que queremos representar?

Para responderla, hicimos una implementación local en Python, pruebas automáticas y una calibración con datos sintéticos. Con este trabajo buscamos reunir evidencia para decidir si podemos usar el simulador como base para comparar escenarios y qué comprobaciones nos faltan antes de aplicarlo a un sistema real.

2. Modelo y supuestos

Definimos una cola única y servidores inicialmente libres. Cada solicitud tiene un identificador, un instante de llegada, un tiempo de servicio y una prioridad. Registramos cuándo empieza y termina su atención, cuánto espera y qué servidor la atiende.

Para el caso principal usamos FIFO: atendemos las solicitudes por

orden de llegada. Si llegan al mismo tiempo, respetamos el orden en que aparecen en los datos. En esta modalidad, el campo de prioridad no cambia el orden.

Trabajamos con estos supuestos:

- Usamos unidades de tiempo abstractas, sin asignarles minutos ni segundos.
- Consideramos servidores idénticos y tiempos de servicio positivos.
- No incluimos fallas, reintentos, abandonos ni pérdidas de solicitudes.
- No interrumpimos una atención que ya empezó.
- Atendemos todas las solicitudes hasta vaciar el sistema.

Calculamos la espera como `inicio - llegada` y la latencia como `fin - llegada`. Para la utilización y el throughput consideramos el tiempo desde cero hasta la última finalización, incluyendo los períodos ociosos.

3. Verificación de la implementación

Primero ejecutamos el caso pequeño del manual con un servidor y dos solicitudes. Como ambas tienen prioridad normal, las atendimos en orden FIFO.

Solicitud	Llegada	Servicio	Inicio	Fin	Espera
1	0	2	0	2	0
2	1	2	2	4	1

Comprobamos que la primera termina en `t = 2` y que la segunda espera una unidad, porque llega cuando el servidor todavía está ocupado. También verificamos que en ambas se cumple `fin >= inicio >= llegada`. Las tres comprobaciones del manual dieron `True`.

Además, hicimos una función para comprobar invariantes, es decir, condiciones que siempre deberían cumplirse. Revisamos que cada

solicitud aparezca exactamente una vez, que los tiempos de atención y espera sean coherentes y que un servidor no atienda dos solicitudes al mismo tiempo. Mantuvimos esta función separada para poder usarla tanto al ejecutar el programa como dentro de los tests.

Ejecutamos **25 tests automáticos y todos pasaron**. Incluimos casos con llegadas simultáneas, períodos ociosos, varios servidores, datos inválidos y distintas reglas de atención. También contrastamos una secuencia con la referencia de la práctica 4 y un caso de llegadas y servicios constantes con su solución matemática.

Para ampliar las comprobaciones, usamos diez semillas fijas con cien solicitudes por semilla, tres cantidades de servidores y ambas reglas de atención: sesenta escenarios en total. No encontramos infracciones de los invariantes. Introdujimos también errores a propósito en algunas trazas y comprobamos que el detector los reconociera.

Con estas pruebas verificamos que la implementación responde como esperamos en los casos ensayados. No tomamos el resultado de los tests como una garantía de que cualquier situación posible esté cubierta.

4. Calibración y comparación con referencias

Distinguimos dos tareas: al **verificar**, comprobamos que el código respete las reglas del modelo; al **validar**, buscamos comprobar que el modelo represente adecuadamente el sistema que queremos estudiar.

Como el manual no proporciona mediciones reales para esta práctica, preparamos una referencia sintética calculada manualmente. Usamos ocho solicitudes distribuidas en dos secuencias separadas: cuatro para calibrar y cuatro para comprobar el ajuste. En ambas usamos un servidor FIFO.

Construimos la referencia con tiempos de servicio un 25 % mayores que los valores iniciales. Por ejemplo, donde el modelo supone que una atención tarda 2 unidades, la referencia indica que tarda 2,5. Esa diferencia corresponde a la duración de la atención; además, puede aumentar la espera de las solicitudes siguientes.

Para calibrar, ajustamos un único factor multiplicativo sobre los tiempos de servicio. Lo calculamos dividiendo la suma de las duraciones de atención de referencia por la suma de los servicios base, usando únicamente el conjunto de calibración. Obtuvimos un **factor de 1,25**. Conservamos las llegadas, la cantidad de servidores y la regla FIFO.

Medimos el error absoluto de cada finalización como `|fin simulado - fin de referencia|`. Si simulamos que una solicitud termina en 4 y la referencia indica 5, el error es una unidad de tiempo. Después calculamos el MAE, que es el promedio de esos errores absolutos.

Conjunto	MAE antes del ajuste	MAE después del ajuste
Calibración	0,8125	0
Validación, separado del ajuste	0,7500	0

También comparamos los instantes de inicio. Después del ajuste, tanto los inicios como los finales coincidieron con la referencia dentro de la tolerancia de `1e-9` unidades de tiempo.

El factor de servicio fue el parámetro que ajustamos en este experimento. Al modificarlo cambiaron las finalizaciones y, cuando había solicitudes esperando, también sus inicios. No comparamos su influencia con la de todos los demás parámetros, por lo que no podemos afirmar que siempre sea el más importante.

Obtuvimos error cero porque construimos la referencia con ese mismo factor. Con esto comprobamos que el procedimiento recupera el ajuste conocido y funciona en otra secuencia que no usamos para calibrar. No demostramos todavía que represente una plataforma real: para eso necesitaríamos registros observados y un margen de error adecuado para esos datos.

5. Desafío adicional: atención por prioridades

Como cambio estructural, comparamos FIFO con una regla que atiende primero a las solicitudes de prioridad alta que estén esperando.

Mantuvimos el mismo servidor, las mismas llegadas y los mismos servicios.

Solicitud	Llegada	Servicio	Prioridad
1	0	4	Normal
2	1	3	Normal
3	2	1	Alta

Con FIFO obtuvimos el orden **1 → 2 → 3**. Con prioridades obtuvimos **1 → 3 → 2**. La solicitud 3 no pasa antes que la 1 porque, cuando llega, la 1 ya está siendo atendida. Aplicamos una prioridad no expropiativa: elegimos quién sigue entre quienes esperan, sin interrumpir el servicio en curso.

Para implementar este cambio tuvimos que modificar la selección de la siguiente solicitud. Cambiamos una regla del modelo, no solamente el valor de un parámetro.

Indicador	FIFO	Prioridades
Latencia promedio	5,33	4,67
P95 de latencia	6,00	6,70
Espera de la solicitud alta	5,00	2,00
Espera promedio de las normales	1,50	2,00

Nos llamó la atención que la latencia promedio bajara mientras el P95 subía. Vimos que mejorar un promedio no implica mejorar todos los resultados: la solicitud alta espera menos, pero postergamos a la solicitud normal que estaba en la cola.

Con estos resultados, consideraríamos la regla por prioridades si el objetivo fuera reducir la espera de las solicitudes altas y aceptáramos el costo para las normales. No la recomendaríamos como una mejora general basándonos solamente en este caso de tres solicitudes.

6. Evidencia y ejecución

Guardamos las trazas, las comparaciones antes y después de calibrar y los resultados del desafío en archivos CSV dentro de `resultados/`. También generamos `evidencia_validacion.json`, donde registramos el factor obtenido, el criterio de aceptación, las comprobaciones y sus limitaciones.

Para ejecutar el programa usamos:

```
python3 "Práctica 11/practica_11_validacion.py"
```

Para ejecutar los 25 tests usamos:

```
python3 "Práctica 11/test_practica_11_validacion.py"
```

Separamos ambos comandos porque el programa principal ejecuta los escenarios y guarda evidencia, mientras que el archivo de tests reúne las pruebas automáticas más amplias. No necesitamos instalar librerías externas.

7. Limitaciones

Trabajamos con datos sintéticos y pocos casos de referencia. Aunque verificamos invariantes en más escenarios, la comparación con referencias quedó limitada a una cola FIFO de un servidor.

Tampoco representamos fallas, abandonos, reintentos ni cambios de capacidad. El ajuste supone que todas las duraciones se corrigen con el mismo factor; con mediciones reales podríamos necesitar otro tratamiento. La tolerancia numérica que usamos sirve para estas referencias exactas, pero no contempla el ruido de medición.

En el desafío estudiamos un caso pequeño. No medimos qué pasaría con llegadas continuas de prioridad alta, que podrían postergar mucho a las normales. Por eso limitamos nuestras conclusiones a las condiciones y escenarios que probamos.

8. Conclusiones

Hicimos una cola ejecutable en Python y reunimos evidencia de que cumple las reglas definidas en los casos ensayados. Los 25 tests pasaron, el caso manual coincidió con lo esperado y no encontramos infracciones de los invariantes en las trazas comprobadas.

Concluimos en que verificar el código y calibrar sus parámetros son pasos distintos. Podemos implementar correctamente una regla y, aun así, obtener diferencias con una referencia si los tiempos de servicio que suponemos no son adecuados. En nuestro ejemplo recuperamos el factor 1,25 y eliminamos esas diferencias, pero la coincidencia con datos sintéticos no reemplaza la validación con datos reales.

También vimos que la regla de atención cambia a quién beneficia el sistema: priorizar una clase reduce su espera, pero puede perjudicar a otra. Antes de usar el modelo para recomendar una política, necesitamos definir qué resultado queremos mejorar y comprobarlo con datos que representen el sistema estudiado.